

regaudit-fhe

Depth-Tracked Regulatory Audit Primitives
for Privacy-Preserving AI Audits

VaultBytes Innovations Ltd
b@vaultbytes.com | vaultbytes.com

2026-11-01 | version 0.2.1 | AGPL-3.0-or-later

AI-assisted engineering work. Not an audit. No production assurance is claimed.

Abstract

regaudit-fhe is an open-source Python library that has six audit primitives: fairness, drift, calibration, provenance, concordance, and cross-model disagreement. Each primitive has a plaintext version and a version written for the CKKS encryption scheme at a multiplicative depth of six with no use of bootstrapping. The output is wrapped in a signed report that can be verified using keys the verifier has enrolled itself. This paper describes version 0.2.1.

The default execution path is plaintext. A signed report would show which key signed it and that it was not changed after. This would not prove that an encrypted computation was done. It would not say that the audit is correct or that anything is compliant. The encrypted path shipped today allows us to show that the encrypted circuits are the same as the plaintext ones by evaluating and decrypting in one process. This is not yet a deployment where the data holder and the evaluator are separate parties.

1. Problem

Several laws and guidance documents request organizations for evidence about automated decision systems. Table 1 shows the ones this library was written with in mind taking into account the status of each on 2026-11-01. They do not all ask for the same thing. NYC Local Law 144 asks for an annual independent bias audit. The EU AI Act sets data governance and accuracy duties for high-risk systems. The model risk guidance for US banks is risk-based guidance and it is not a rule for audits at run time.

Four entries in Table 1 have changed since the previous version of this paper. SR 11-7 was superseded in April 2026 [1]. The Colorado AI Act was repealed and reenacted in May 2026 as a law about automated decision-making technology [2]. CFPB Circular 2022-03 was withdrawn in May 2025 [3] but the adverse action duties in ECOA and Regulation B are still in place. The dates of the EU AI Act for high-risk systems changed in July 2026 [4].

Table 1 is a summary and it is not meant to be legal advice. Anyone relying on these instruments would need to check the primary text.

The data these instruments are used on is sensitive. This can include protected attributes and per-row labels and predictions. This can also include health data and training-row identifiers. Sending this data to a third-party auditor can be conflicting with privacy law and internal policy. Keeping this data in-house would make it difficult for independent checking. regaudit-fhe is made to fill this gap between these two positions. This allows us to calculate the scalars an auditor would want and sign the result in a way that allows a third party to check.

The library does not discharge any obligations through any of these instruments. A compliance professional would use its output as one input for this decision.

2. What a Report Does and Does Not Show

Four questions need to be separated before looking at the design. A verdict from the library would only solve the first two questions. The first question would be if the report is intact. The Ed25519 signature

Table 1: Instruments this library was written with in mind, status on 2026-11-01. A summary only, not legal advice.

Instrument	Status and what it asks for
NYC Local Law 144 of 2021 [9]	In force, enforced since 5 July 2023. An annual independent bias audit of automated employment decision tools. The rules use selection rates and impact ratios, and `audit_fairness` reports differences, so its scalars do not replace that calculation.
EU AI Act, Regulation (EU) 2024/1689 [10], amended by (EU) 2026/1744 [4]	High-risk duties now apply from 2 December 2027 (Annex III) and 2 August 2028 (Annex I). Art. 10 data governance, which was amended, Art. 15 accuracy, robustness and cybersecurity, Art. 72 post-market monitoring.
Colorado SB26-189 [2]	Repealed and reenacted SB24-205, signed 14 May 2026. Automated decision-making technology, with requirements starting 1 January 2027. The legislative summary describes documentation, notices, records and consumer rights, and does not describe impact assessments or bias audits.
ECOA and Regulation B	Adverse action duties stay in place. CFPB Circular 2022-03 was withdrawn on 12 May 2025 [3]. The primitives do not produce adverse action reasons.
FDA guidance on predetermined change control plans [11]	Final guidance, December 2024. Marketing submission recommendations for AI-enabled device software.
SR 26-2 and OCC Bulletin 2026-13 [1]	Revised model risk guidance, 17 April 2026, superseding SR 11-7 and OCC Bulletin 2011-12. Generative and agentic AI are outside its scope.
EMA reflection paper [12]	Reflection paper on AI in the medicinal product lifecycle, 2024.
ISO/IEC 23053:2022 [13]	A framework standard, not a regulation.
BCBS 239 [14] and BCBS d515 [15]	Risk data aggregation (2013) and sound management of operational risk (revised 2021). General banking principles, not specific to AI.

and the content hash would help us know this. The second question would be if the signer is someone that the verifier trusts. A key that the verifier enrolled in its own trust store would be needed for this. A report would have the public key that would check its signature. Any person can create a key and sign a report and add the key. This report would be internally consistent and would not prove anything about who has signed it.

The third question is if the signed provenance agrees with itself meaning that the signed backend, `parameter_set` and `parameter_set_hash` are not contradictory. This is a bug detector and not a defence for a dishonest signer. The fourth question is if the computation really ran under FHE. The library will not give us this. The backend field is what the signer claimed and a default call to the library would sign plaintext even on the encrypted path.

A `TRUSTED` verdict would mean that the report was signed with a key in the verifier's own trust store and is within the scope of that key. All the policy rules were met. This would not mean that the audit is correct and it would not mean that anything is compliant.

3. Architecture

1. The plaintext model in `regaudit_fhe._slot` is a numpy slot vector with a depth tracker. It raises `DepthBudgetExceeded` when a circuit goes beyond its budget.

2. The six audit primitives are `audit_fairness`, `audit_provenance`, `audit_concordance`, `audit_calibration`, `audit_drift`, and `audit_disagreement`. Each has a plaintext oracle and a depth-budgeted circuit with the same semantics.

3. The encrypted backends are optional. The `[fhe]` extra adds TenSEAL. The `regaudit_fhe.fhe` module has an encrypted function for each primitive. The `[openfhe]` extra adds an experimental OpenFHE adapter that runs five of the six primitives. Encrypted concordance is pending on OpenFHE due to the need for rectangular matrix products.

4. There are two formats for signed reports: `report.v1` and `report.v2`. Each has canonical JSON, SHA-256 digest, an Ed25519 signature, a `parameter_set_hash` and optional `input_commitments`. An optional block can be added to show differential-privacy noise in a result.

5. The trust and verification layer contains the trust store, key status, `verify_trusted`, the verification

profiles and the replay cache.

6. The surfaces are 14 JSON Schemas, a CLI that has the commands audit, keygen, trust, verify, serve, and schema, and a reference FastAPI server.

3.1. How the encrypted path runs today

```
Subject  --[CKKS public key]--> Subject encrypts inputs locally
                                          v
                                Operator runs the encrypted circuit
                                          v   (depth <= 6, no bootstrap)
                                  Encrypted result + signed report
                                          v
                                Auditor decrypts the result
                                          v
                                Plaintext-side threshold compare
                                + report verification
```

Figure 1: The intended deployment. The library does not implement this separation today.

The intended deployment will have a Subject that has the data and encrypts it. It will also have an Operator that will evaluate the circuit and an Auditor that will decrypt the result. The library does not implement this separation today. In the functions shipped such as `fairness_encrypted`, the context will have the secret key. A call will encrypt the inputs, evaluate the circuit and decrypt the aggregate scalars. This means that the same process will see both the inputs and the result.

A deployment that wants the separation would need to build it. The evidence for it would not come from this library.

The CLI and the HTTP server have no encrypted path. They run the plaintext primitives on the JSON received and sign plaintext as the backend. The server is therefore not a privacy boundary. Encrypted execution is only available through the Python API with the `[fhe]` extra.

4. Threat Model

The threat model has four roles. These roles can be one legal entity but would be treated separately. The Subject is the owner of the plaintext per-row data. The Auditee has the model that is being audited. The Operator runs the circuit. The Auditor has the verifying keys and the CKKS secret key for decrypting. As described in the section above the shipped encrypted functions have the Operator and the Auditor in the same process.

4.1. Security of the encryption

The parameter sets are sized for 128-bit IND-CPA security under Ring-LWE. `CKKSParams` enforces the bounds from the HomomorphicEncryption.org standard [5]. The bounds are 438 bits for $N = 2^{14}$ and 881 bits for $N = 2^{15}$. This is a parameter-level claim. An adopter would need to check this with a lattice estimator [6] for their own library versions and deployment. They would need to make sure that this is trustworthy before relying on it.

IND-CPA does not cover how this library uses CKKS. Li and Micciancio showed how a passive attacker can get the CKKS secret key from seeing decryption outputs [7]. Later, Guo et al. were able to get a key from one shared decryption output in OpenFHE when the noise flooding used average-case noise estimates [8]. regaudit-fhe decrypts aggregate scalars and places them in the signed result with no rounding and no noise flooding.

If a party has the ciphertexts from an evaluation and can see the result of it then this attack setting may apply. The Operator would be this party in the intended deployment. IND-CPA-D is a property of a scheme under a defined decryption oracle. The claims for a deployed system would depend on who can see which ciphertexts and what outputs. The library makes no claim for IND-CPA-D and its parameter sets have not been analyzed for it.

A deployment that separates the roles would need to treat this as an open problem. It should not show decrypted values to the Operator.

An earlier version of the threat model said that collusion between Operator and Auditor adds no new leakage. In the intended deployment they would together hold the ciphertexts and the secret key.

Scope of the key-recovery experiment

A limited key-recovery experiment was conducted against the library's evaluated CKKS parameter configurations and output shapes. The tested attack succeeded in a separate setup that exposed information not provided by the library's tested outputs. The same attack did not recover the secret key from the library's tested output shapes in the runs performed.

These observations are limited to the attack implementation, configurations, output shapes, and trials examined. They do not establish resistance to other attacks or provide a general IND-CPA-D security guarantee. The library's confidentiality claims remain conditional on its stated threat model, supported parameter configurations, and operational assumptions. The runs used fresh ciphertexts and one seed per setting, and not the output of a real audit circuit. The only library configuration tested was $N = 2^{14}$ with the default 400-bit chain.

4.2. Depth and report integrity

Depth is restricted in two places. `CKKSParams` would not allow a depth outside 1 to 6 or a modulus chain that is too short for the declared depth. The depth tracker would raise an error if a circuit goes over its budget. No primitive uses bootstrapping.

The depth tracker counts operations and is not a reading of the modulus levels. We have taken measurements of the levels used when $N = 2^{14}$ in the default chain. Concordance used 6 levels against a declared 5 and calibration used 5 against a declared 4. The check did not refuse these since the counter has read 5 and 3.

The SHA-256 digest and the Ed25519 signature cover the canonical form of the signed body. Any change in a signed value would change the digest. Changes in whitespace, key order, and string escapes would not change the digest. These values are not part of the canonical form. The `input_commitments` field would hold SHA-256 digests of the inputs when the caller provides them. A verifier that has the inputs can check them. The commitments would not show that these inputs went through an encrypted circuit.

4.3. Out of scope

Side channels in the host, Subjects that would create inputs to get a wrong decision, Auditors that might leak decrypted values, long-term cryptanalytic advances and SHA-256 collisions are out of scope. An Operator that has control over the signing key is also out of scope, since a signature will only show that the key was used. These would be part of the deployment layer.

5. Six Primitives

Each primitive has a plaintext oracle, a depth-budgeted circuit, an encrypted TenSEAL function and input and output JSON Schemas. Table 2 shows the depth each one declares. The module names are internal and the public API uses descriptive names. The notes below explain what each encrypted function encrypts and decrypts, as read from the code for this release.

Only `y_pred` is encrypted for the function `fairness_encrypted`. `y_true` and the two group masks are plaintext in the same call. An earlier version of the threat model showed all of them as encrypted, which did not match the function. The group sizes and label-positive counts are public to the Auditor. Therefore, we can make the reciprocals plaintext scalars. No encrypted division is needed. The disparity scalars will be decrypted and the decision on the breach will be made in the plaintext side.

For `audit_provenance` the attribution values are encrypted. The row identifiers are plaintext and the hash function and seed are public. A verifier would be able to reproduce the bucket assignments. The bucket aggregates would be decrypted.

For `audit_concordance` the risk, time and event vectors are encrypted. Three aggregates are decrypted and the C-index is computed from them on the plaintext side. The previous version of this paper said two.

For `audit_calibration` the scores are encrypted and the per-class quantile thresholds are public to the Auditor. The function decrypts a membership signal for each class and turns it into a bitmask on the

Table 2: Primitives and declared depth on the TenSEAL backend. Declared depth is the budget each function asserts.

Module	API	Depth	Use
`egf_imss`	`audit_fairness`	4	Demographic parity, equal opportunity and predictive parity differences between two groups.
`etk_fpa_hbc`	`audit_provenance`	3	Top-K training-data provenance histogram through hashed buckets.
`esc_cia`	`audit_concordance`	5	Harrell C-index. The plaintext model has depth 4.
`ecp_qssp`	`audit_calibration`	4	Conformal prediction-set bitmask from public quantile thresholds.
`ew1_cdsf`	`audit_drift`	2	Squared L2 distance between two CDFs. The plaintext model has depth 1.
`ecmd_jps`	`audit_disagreement`	5	Pairwise variance across $M \geq 3$ degree-3 polynomial surrogates.

plaintext side. Therefore, the decrypted values would be the polynomial outputs and not just the final bits. This decrypted signal shows the scores. In a test of 4 scores they could be worked out from it with an error of about 4e-6.

For `audit_drift` the circuit would calculate the squared L2 distance between two CDFs. This would be a Cramer-von-Mises style statistic. This was chosen over the strict use of the W1 distance to avoid the use of an encrypted absolute value. This would be zero only if the two distributions are equal. There would also be a `w1_distance` field. This will be computed in the plaintext for reference only. The depth of the plaintext model would be 1 but for use with TenSEAL we would need depth of 2. This is due to the lack of a public rotation for the `CKKSVector` and the use of a plaintext matrix product for the prefix sum.

For `audit_disagreement` the input vector will be encrypted. Each of the $M \geq 3$ model versions will be a public degree-3 polynomial surrogate. The average variance will be decrypted as well as one summary mean for each model.

The approximation polynomials have low degrees but have a large error near the decision boundary. The sign polynomial is $p(x) = 1.5x - 0.5x^3$. Its error is 31.25% when $|x| = 0.5$ and it gets worse when closer to zero. It is not a minimax fit, which the previous version of this paper said it was. The reciprocal polynomial is a Taylor expansion of $1/x$ at $x = 1$. It has a worst-case relative error of 6.25% when $0.5 < x < 1.5$. A result close to a threshold would need more care than what Table 4 suggests.

6. CKKS Parameters and Measurements

`CKKSParams` is the validated parameter set. Table 3 shows the defaults. The chain has 8 primes and has a total of 360 bits. The previous version of this paper gave $\log Q$ as about 240. This was incorrect. The `build_d6_context` function has its own defaults where $N = 2^{14}$ and the chain is of 9 primes totaling 400 bits. A caller that does not pass parameters will get a different set than Table 3. The benchmark passes the 360-bit chain in both ring sizes.

Table 3: `CKKSParams` defaults.

Field	Default	Meaning
`ring_dim`	2^{15}	Power of two. At least 2^{14} is required for this depth.
`multiplicative_depth`	6	Must lie in $[1, 6]$.
`scaling_mod_size`	40	Bit size of each interior prime.
`first_mod_size`	60	Bit size of the first and last primes.
`security_level`	`HEStd_128_classic`	At least 128-bit IND-CPA.
`coeff_mod_bit_sizes`	$\langle 60, 40, 40, 40, 40, 40, 40, 60 \rangle$	8 primes, 360 bits in total. The bound is 881 bits at 2^{15}.
`rotation_steps`	$\pm 2^k$	Powers of two up to the slot count.
`precision_loss_bound`	10^{-2}	Worst-case rescale loss bound.

The library declares that TenSEAL is the reference backend and needs version 0.3.16 or later. It does not configure the key-switching decomposition and therefore the previous statement that `dnum = 3` is removed. The use of the OpenFHE adapter is experimental and was not part of these measurements.

The benchmark in `benchmarks/bench_fhe.py` was run again on the 0.2.1 wheel from PyPI with TenSEAL 0.3.18. Table 4 shows the median runtime of 3 repetitions and the largest absolute error compared to the plaintext oracle. These numbers come from one run where a laptop was also running other work. The inputs are small with batches of 64 for fairness and provenance and 16 for the others. So they only show the order of size. The process memory was about 3.7 GB when $N = 2^{15}$. This script reads memory for the entire process. We cannot get a figure for each primitive.

Table 4: Measured on the 0.2.1 wheel. Apple M3 Max, 36 GB, macOS 26.6.2, Python 3.12.11, NumPy 2.5.4, TenSEAL 0.3.18. Median of 3 repetitions. The error is the largest absolute difference from the plaintext oracle at $N = 2^{15}$. Calibration compares the final bitmask, so its error is not an error on the polynomial output.

Primitive	Batch	Depth	2^{14} (s)	2^{15} (s)	Max error	Flip rate
fairness	64	1 / 4	0.76	1.31	7.0×10^{-7}	0 of 10
provenance	64	1 / 3	1.94	3.41	1.6×10^{-5}	not measured
concordance	16	5 / 5	10.83	15.27	1.4×10^{-6}	not measured
calibration	16	3 / 4	0.06	0.08	0	not measured
drift	16	2 / 2	0.48	0.82	3.6×10^{-6}	0 of 10
disagreement	16	3 / 5	0.39	0.66	1.5×10^{-8}	0 of 10

```
python benchmarks/bench_fhe.py --rings 14 15 --reps 3 --flip-trials 10
```

The threshold flip rate is the fraction of trials where CKKS noise changes a breach decision relative to the plaintext oracle. This was 0 out of 10 trials for fairness, drift, and disagreement in both ring sizes. This is weak evidence and would be consistent with a true rate of up to about 26% at 95% confidence. The script does not measure the rate for provenance, concordance, and calibration and it writes 0 for them. So the 0.00% across all primitives in the previous version of this paper said more than was measured.

7. Reports and Verification

`report.v1` is the original format and `report.v2` was added for strict verification. `report.v2` adds signed `audit_run_id`, `deployment_id`, `key_id`, `public_key_fingerprint` and `trust_policy_version` fields. It also allows for an optional audience. Figure 2 shows a `report.v2`. The CLI will write a `report.v2` if given a long-lived key with `audit --signing-key`. The HTTP endpoints will continue to write `report.v1` using an ephemeral key. This report will never be more than `SELF_SIGNED_ONLY`.

```
{
  "schema":                 "regaudit-fhe.report.v2",
  "schema_version":         "regaudit-fhe.report.v2",
  "trust_policy_version":   1,
  "algorithm_version":      "0.2.1",
  "primitive":              "fairness",
  "backend":                "plaintext",
  "parameter_set":          { "backend": "plaintext", "library_version": "0.2.1", ... },
  "parameter_set_hash":     "9f6e012d6c51ca88...",
  "regulations":            ["NYC_LL144", "EU_AI_ACT_ART10", ...],
  "result":                 { "demographic_parity_diff": 0.5, ... },
  "input_commitments":      [{ "name": "group_a", "sha256": "9438b00c..." }, ...],
  "depth_budget":           { "declared": 6, "consumed": 6 },
  "audit_run_id":           "c0ad3a73-f267-4acf-8381-7361ef78329b",
  "deployment_id":          "d1",
  "audience":               ["v1"],
  "issued_at":              "2026-11-01T20:07:47.057506+00:00",
  "issuer":                 "acme",
  "key_id":                 "k1",
  "public_key_fingerprint": "84cc09f8d95b60b7...",
  "receipt":                { "sha256": "...", "signature_alg": "Ed25519", ... }
}
```

Figure 2: An abridged `report.v2` written by `regaudit-fhe audit --signing-key` in version 0.2.1.

`verify_trusted` is the main entry point. It would parse the report strictly and would need the trust store. It would return `TRUSTED`, `SELF_SIGNED_ONLY` or `INVALID`. Only the `TRUSTED` would be an acceptance. The result would refuse any truth value. This would make sure that a caller cannot accidentally consider

any verdict true. The `verify_receipt` and `verify_envelope` functions remain for compatibility but can return true for any intact report signed by any person. The previous version of this paper pointed to `verify_envelope` but that should not be followed.

Strict parsing would not allow for duplicate keys in JSON, non-finite numbers, unknown or missing members in a `report.v1` or contradictory provenance. `report.v2` has a separate strict canonical form. The verifier would use its key from its trust store and not the one in the report. A trust store entry would link a `key_id` and public key to an issuer name. It would also have a status and validity dates. A parameter-set pin can be added.

Trust stores are local files. There is no authoritative revocation service. The CLI has no retire, revoke or rotation commands in this release. A trust store in the older `key_id` to PEM form has no issuer binding. A registered key can sign as any issuer and it would still report `trusted_issuer` as true. Thus, the entries form should be used.

Table 5 shows the verification profiles. The high-assurance profile requires a trust store and a maximum age. It also does replay checking. It would refuse a report without an audience naming the verifier. It would accept a local unauthenticated trust store and a local replay file. This means that the name would represent stricter settings. It would not yet mean the authenticated and fresh store needed for this profile as described in the trust specification.

Table 5: Verification profiles as implemented in 0.2.1.

Profile	Trust store	Replay	report.v1
`development`	Optional. Self-signed checking allowed on loopback.	Off by default.	Allowed
`confidential`	Required.	Required, persistent.	Refused
`high-assurance`	Required. A local file is accepted today.	Required, persistent. A local file is accepted today.	Refused

Replay detection is local to the replay store a verifier is configured with. The cache will remember the `key_id` and `audit_run_id` pair at verification time and refuse it if it still has it. This is not a global replay guarantee. Verifiers or server instances that do not share a store can each accept the same report once. A report that is forgotten if the store is deleted, reset, rolled back, or restored from a backup can be accepted again. The library would not detect this.

An in-memory cache will be lost if the system is restarted.

Retention is fixed by the policy in force when the id was recorded. A way to allow a report to be accepted a second time would be by increasing the `max_age` for a store in use. An operator should not raise `max_age` on a store in use. Starting a new replay file would not help since it would remember nothing.

The `issued_at` field is a claim written by the signer. A timestamp member on a `report.v1` covers the body and not the signature. Therefore, it cannot be used as evidence of when the report was signed. A timestamp on a `report.v2` would count as evidence if the caller's pinned verifier checks it. The library does not bundle a timestamp authority.

Three signed fields need care. The `depth_budget` consumed value will default to 6 unless passed a measured value. So a default report would not show what the depth used is. The `parameter_set_hash` would include the version of the library as well as the CKKS parameters. Therefore, a parameter-set pin would allow reports from one library version only. The regulations field would default to a list of labels for each primitive. Some labels would refer to instruments in Table 1 that have since changed. These labels are not a claim of compliance.

8. Server and Release Pipeline

The HTTP server is a reference deployment. It has limits on the size of the body, in-flight body bytes, concurrency and request time. A body that stalls for 10 seconds will get a 408 response. `/v1/verify` will refuse bodies deeper than 16 or wider than 65,536 elements before parsing. These are availability controls. A reverse proxy to limit body size, connections, timeouts, and rates would be needed. TLS needs to end at this layer.

Releases are built through a workflow that only runs on a version tag. It builds the wheel and sdist. It runs tests on the wheel installed in Python 3.10 to 3.13 with tests from the sdist. It runs pip-audit

and generates an SBOM. It signs the files with Sigstore. The upload is held for approval in a protected environment. PyPI attestations are created during the upload. With one maintainer, the approval would be a second click from the same person and not a separation of duties. The evidence for this release is in `docs/releases/RELEASE_0.2.1_RECORD.md`.

9. Limitations and Status

This release relies on AI-assisted testing and review. It has had no independent external security audit and no claim is made for production assurance. No regulator has endorsed the report format. This package is pre-alpha and public APIs may change in minor versions.

Signed revocation data, key rotation, and rollback protection are not implemented in this release. The design and test work on these features are ongoing and have no committed release date. A verifier can only know that a key is revoked through the use of its local trust store. Python 3.15, native Windows, the `[fhe]` Docker image, and the OpenFHE backend are not qualified. Post-quantum assurance is not claimed beyond the standard Ring-LWE assumptions. $N = 2^{16}$ is not in the published measurements.

Companion documents

- Trust and verification specification: `docs/TRUST_SPEC.md`
- Threat and leakage model: `docs/THREAT_MODEL.md`
- Scope statement: `COMPLIANCE.md`
- Deployment guide: `docs/DEPLOYMENT.md`
- End-to-end walkthrough: `docs/WALKTHROUGH.md`
- Supply-chain controls: `docs/SUPPLY_CHAIN.md`
- Release record: `docs/releases/RELEASE_0.2.1_RECORD.md`
- Per-primitive specifications: `docs/specs/`
- OpenFHE design note: `docs/roadmap/openfhe_backend.md`

References

[1] Board of Governors of the Federal Reserve System, SR 26-2, *Revised Guidance on Model Risk Management*, 17 April 2026; OCC Bulletin 2026-13, *Model Risk Management: Revised Guidance*, 17 April 2026.

[2] Colorado General Assembly, SB26-189, *Automated Decision-Making Technology*, signed 14 May 2026.

[3] Consumer Financial Protection Bureau, *Interpretive Rules, Policy Statements, and Advisory Opinions; Withdrawal*, 90 FR 20084, 12 May 2025.

[4] Regulation (EU) 2026/1744 (Digital Omnibus on AI), in force 27 July 2026.

[5] M. Albrecht et al., *Homomorphic Encryption Standard*, HomomorphicEncryption.org, version 1.1, 2018. See also J.-P. Bossuat et al., *Security Guidelines for Implementing Homomorphic Encryption*, IACR ePrint 2024/463.

[6] M. Albrecht, R. Player and S. Scott, *On the concrete hardness of Learning with Errors*, Journal of Mathematical Cryptology, 2015, and the Lattice Estimator.

[7] B. Li and D. Micciancio, *On the Security of Homomorphic Encryption on Approximate Numbers*, EUROCRYPT 2021, IACR ePrint 2020/1533.

[8] Q. Guo, D. Nabokov, E. Suvanto and T. Johansson, *Key Recovery Attacks on Approximate Homomorphic Encryption with Non-Worst-Case Noise Flooding Countermeasures*, USENIX Security 2024.

[9] New York City Department of Consumer and Worker Protection, Local Law 144 of 2021, *Automated employment decision tools*.

[10] Regulation (EU) 2024/1689 of the European Parliament and of the Council (Artificial Intelligence Act).

[11] U.S. FDA, *Marketing Submission Recommendations for a Predetermined Change Control Plan for Artificial Intelligence-Enabled Device Software Functions*, final guidance, December 2024.

[12] European Medicines Agency, *Reflection paper on the use of artificial intelligence in the medicinal product lifecycle*, 2024.

[13] ISO/IEC 23053:2022, *Framework for AI systems using machine learning*.

[14] Basel Committee on Banking Supervision, *Principles for effective risk data aggregation and risk reporting*, BCBS 239, 2013.

[15] Basel Committee on Banking Supervision, *Revisions to the principles for the sound management of operational risk*, BCBS d515, March 2021.

---

regaudit-fhe is licensed under AGPL-3.0-or-later.
Source: `github.com/BAder82t/regaudit-fhe`.
PyPI: `pypi.org/project/regaudit-fhe`.
Maintained by VaultBytes Innovations Ltd; contact `b@vaultbytes.com`.